

PINN

¡Vamos con una PINN paso a paso, con cuentas y sin código! Tomo una EDO 1D muy simple para ver todo con lápiz y papel.

1) ¿Qué es una PINN?

Una **Physics-Informed Neural Network (PINN)** aproxima la solución $u(x)$ de una ecuación diferencial imponiendo en la **función de pérdida**: (i) que el **residuo** de la ecuación sea pequeño en puntos interiores (“collocation”), y (ii) que las **condiciones de contorno**/iniciales se satisfagan.

2) Problema base (EDO 1D)

Considera en (x) : $[u'(x)+u(x)=0, u(0)=1.]$ La solución exacta es $(u(x)=e^{-x})$. La usaremos sólo como referencia.

3) Aproximador neuronal

Usa una red muy simple (una sola neurona oculta, para poder derivar a mano): $[u(x)=v,(wx+b)+c, =(v,w,b,c).]$ Derivada: $[u'(x)=v,w,^2(wx+b).]$

Residuo de la EDO (lo que debería ser 0): $[r(x):=u'(x)+u(x) = v w,^2(s);+;v,(s);+;c,s:=wx+b.]$

4) Pérdida (PINN “suave” con penalización)

Con (N) puntos interiores $(\{x_k\})$ y una penalización (>0) para la condición inicial: $[L() =\{k=1\}^N r(x_k)^2 ;+;,(u_0-1)^2, u_0=v,(b)+c.]$

5) Gradientes analíticos (para ver “cómo aprende”)

Es útil anotar las derivadas parciales del **residuo** $(r(x))$ y de la **condición** (u_0) .

Sea $(t:=(s))$ y $(q:=2(s)=1-t^2)$. Entonces $[]$

Para la condición inicial $(u_0=v,(b)+c)$: $[=(b), =0 (x=0), =v,^2(b), =1.]$

Regla de la cadena para (L): $[=\{k=1\}^N r(x_k), ;+;2,(u_0-1),.]$

En práctica, esto lo hace *autodiff*; aquí se ve que la PINN **minimiza** el residuo interior y la violación de la condición inicial.

6) Mini-cálculo numérico en un punto (todo a mano)

Toma parámetros “plausibles” que ya muestran decaimiento: $[v=-1, w=1, b=0, c=1.]$ En $(x=0.5)$: $(s=0.5), (t=(0.5)), (q=^2(0.5)).$

- Aproximación y derivada: $[u_0(0.5)=v,t+c=-0.462117+1=0.537883, u'(0.5)=v w q=-0.786447.]$
- **Residuo**: $[r_0(0.5)=u'(0.5)+u(0.5)=-0.786447+0.537883=-0.248564.]$
- **Gradientes del residuo** en $(x=0.5)$: $[]$
- **Condición inicial**: $(u_0=v(0)+c=1) \Rightarrow$ el término de contorno **no aporta** a (L) (su residuo es 0).

Si (sólo para ilustrar) usas este **único** punto interior $((N\{=1\}), ()$ arbitraria), los gradientes de (L) quedan: $[=2,r,(-0.248564) (1.248564)=-0.620, (-0.248564)(-0.816671)=+0.406,] [(-0.248564)(-0.060447)=+0.030, =2(-0.248564)=-0.497.]$ Una **pasada de descenso** $(-,L)$ movería (w) y (b) **hacia arriba** (signos $(+)$), y (v,c) **hacia arriba** también (porque restas un gradiente negativo), todo con el fin de **reducir el residuo** en el punto.

7) Variante con restricción dura (condición inicial exacta)

Otra táctica frecuente en PINNs: construir un ansatz que **satisfaga la condición** por construcción. Aquí, impón $u(0)=1$ mediante $u(x)=1+xN(x)$, con N una red cualquiera. Entonces **siempre**. El residuo es $R(x)=u'(x)-(1+xN'(x))=N(x)+xN'(x)+1-xN'(x)$ y la pérdida **ya no necesita** término de contorno: $L=\sum_{k=1}^N (x_k)^2$. (De nuevo, N' se obtiene por regla de la cadena—autodiff en práctica—pero aquí ves la **forma exacta** de las derivadas que se minimizan.)

Conclusión

- Una PINN minimiza el **residuo** de la EDO en puntos interiores y, además, **impone** las **condiciones** (suaves o duras).
- El ejemplo muestra, con **fórmulas explícitas**, cómo se computan R , L y sus **gradientes**.
- Con suficientes puntos y una red un poco más rica, el mínimo de L aproxima la solución $u(x)=e^{-x}$ en $[0,1]$.

Si quieres, repetimos el ejercicio para una **EDO de 2º orden** (p. ej. $u''+u=0$ con $u(0)=0$, $u'(0)=1$), donde se ve bonito cómo entra $u''(x)$ en el residuo.